

Performance evaluation of Meta-heuristics techniques for Generalized Assignment Problem : A comparative study



M.Tech CS

Semester : II

Subject: Evolutionary Computation

Submitted by : Kavita Pat Pingua

Roll No : 2025PGCSCS20

Submitted to : Dr. Dinesh Kumar

ABSTRACT	3
1. INTRODUCTION.....	4
1.1 OVERVIEW OF COMBINATORIAL OPTIMIZATION PROBLEMS	4
1.2 GENERALIZED ASSIGNMENT PROBLEM (GAP)	4
1.3 ROLE OF META-HEURISTICS TECHNIQUES IN SOLVING NP-HARD PROBLEMS	4
1.4 OBJECTIVE OF THE STUDY	5
2. BACKGROUND	5
2.1 GENERALIZED ASSIGNMENT PROBLEM (GAP).....	5
2.2 META-HEURISTICS OVERVIEW	6
3. PROBLEM DEFINITION	7
3.1 PROBLEM FORMULATION	7
3.2 NOTATIONS AND VARIABLE DEFINITIONS.....	8
4. COMPARATIVE STUDY OF META-HEURISTIC TECHNIQUES	9
4.1 BINARY-CODED GENETIC ALGORITHM (BCGA)	9
4.2 REAL-CODED GENETIC ALGORITHM (RCGA).....	9
4.3 PARTICLE SWARM OPTIMIZATION (PSO).....	9
4.4 TEACHING-LEARNING-BASED OPTIMIZATION (TLBO)	10
4.5 DIFFERENTIAL EVOLUTION (DE)	10
4.6 ARTIFICIAL BEE COLONY (ABC).....	10
5. EXPERIMENTAL ANALYSIS	11
5.1 PARAMETER SETTINGS.....	11
5.2 CONSTRAINT HANDLING AND VARIATION STRATEGY	13
5.3 RESULTS AND DISCUSSION.....	18
5.4 DISCUSSION	23
6. CONCLUSION	24

Performance Evaluation of Meta-heuristics Techniques for the Generalized Assignment Problem: A Comparative Study

Abstract

Generalized Assignment Problem (GAP) is a combinatorial optimization problem that is NP-hard. It involves giving a group of agents a set of tasks while maximizing total profit within resource capacity limits. Because exact methods are too hard to use on large-scale problems, meta-heuristic approaches are a good choice.

This study implements and evaluates several population-based meta-heuristic algorithms, including the Binary-Coded Genetic Algorithm (BCGA), Real-Coded Genetic Algorithm (RCGA), Particle Swarm Optimization (PSO), Teaching-Learning-Based Optimization (TLBO), Differential Evolution (DE), and Artificial Bee Colony (ABC), using standard GAP datasets (gap8, gap10, gap12). To make sure that solutions are possible, different ways of handling constraints, such as repair, penalty, and replacement strategies, are used.

The experimental setup consists of several independent runs for each algorithm, and the performance is judged by the best fitness values and how quickly they converge. In this case, a comparative analysis using convergence plots and histogram-based evaluation shows the pros and cons of each method.

The results indicate that real-coded approaches, particularly DE and TLBO, achieve superior performance in terms of solution quality and robustness, while PSO demonstrates faster convergence but is prone to premature stagnation. The study concludes that hybrid constraint handling and continuous encoding significantly enhance optimization performance for GAP.

1. Introduction

1.1 Overview of Combinatorial Optimization Problems

Combinatorial optimization problems require identifying an optimal solution from a finite, yet generally extensive, collection of discrete configurations. As the problem size grows, the number of possible solutions grows exponentially, which makes exhaustive search impossible to do with computers. Some examples are problems with scheduling, routing, allocation, and assignment.

These kinds of problems usually have an objective function that needs to be optimized (either maximized or minimized) while following a set of rules. Many combinatorial problems are NP-hard, which means that there are no known polynomial-time algorithms that can guarantee the best solutions. Consequently, approximate and heuristic techniques are extensively employed to derive near-optimal solutions within acceptable computational timeframes.

Meta-heuristic algorithms have become effective tools for addressing these issues because they can efficiently navigate extensive search spaces while maintaining a balance between exploration and exploitation.

1.2 Generalized Assignment Problem (GAP)

The Generalized Assignment Problem (GAP) is a basic combinatorial optimization problem that involves giving a group of agents a group of tasks to do. Each assignment makes a certain amount of money while using up a limited amount of resources that the agent has available. The goal is to maximize total profit possible without going over the limits of any agent's capacity.

The complexity of GAP arises because assignment decisions are combinatorial and there are many constraints.

GAP is useful in many areas, such as logistics, resource allocation, scheduling, and load balancing systems, where limited resources need to be used in the best way possible. This work uses profit matrices, resource consumption matrices, and capacity vectors to show GAP instances. It also uses binary assignment matrices (in BCGA) or real-valued vectors decoded into binary form (in RCGA, PSO, TLBO, DE, and ABC) to show solutions.

Because GAP is so complicated, meta-heuristic methods are used to quickly search the solution space and find high-quality solutions that work.

1.3 Role of Meta-heuristics techniques in solving NP-hard problems

Meta-heuristic algorithms are advanced optimization methods that are meant to quickly search through large and complicated search spaces. They work especially well for NP-hard problems like GAP, where traditional optimization methods don't work well when the problem gets bigger.

These algorithms use population-based or iterative improvement methods and are based on natural processes like evolution (GA), swarm intelligence (PSO, ABC), teaching-learning dynamics (TLBO), and vector-based mutation (DE). This research employs both discrete (BCGA) and continuous (RCGA, PSO, TLBO, DE, ABC) representations to address the GAP.

1.4 Objective of the Study

The goal of this study is to look at how well different meta-heuristic algorithms (BCGA, RCGA, PSO, TLBO, DE, ABC) work for solving the Generalized Assignment Problem when different strategies for handling constraints are used.

The assessment emphasizes solution quality, convergence behavior, and robustness to determine the optimal method for extensive GAP instances.

2. Background

2.1 Generalized Assignment Problem (GAP)

2.1.1 Formal Definition and Mathematical Formulation

The Generalized Assignment (GAP) consists of assigning a set of tasks $i = 1, 2, \dots, m$ to a set of agents $j = 1, 2, \dots, n$. Each assignment yields a profit p_{ij} and consumes a resource r_{ij} from agent j , which has a limited capacity c_j .

The decision variable is defined as:

$$x_{ij} = \begin{cases} 1, & \text{if task } i \text{ is assigned to agent } j \\ 0, & \text{otherwise} \end{cases}$$

The mathematical formulation is:

Objective Function:

$$\max \sum_{i=1}^m \sum_{j=1}^n p_{ij} x_{ij}$$

subject to:

1. Assignment Constraint (each task assigned at most once):

$$\sum_{j=1}^n x_{ij} \leq 1 \quad \forall i$$

2. Capacity Constraint (resource limit of each agent):

$$\sum_{i=1}^m r_{ij} x_{ij} \leq c_j \quad \forall j$$

2.1.2 Explanation of Constraints

Capacity Constraints:

Each agent has a limited capacity c_j , and the total resource consumption of assigned tasks must not exceed this limit. In the code, this is checked using:

```
used = sum(sol[i][j]*resource[i][j])
```

Violations are handled using repair, penalty, or replacement methods.

Assignment Restrictions:

A task should be assigned to at most one agent. While the provided implementation allows multiple assignments in raw solutions, feasibility is enforced indirectly through constraint handling methods, particularly repair and replacement strategies.

Feasibility Maintenance:

Ensuring feasibility is critical, as infeasible solutions may distort fitness evaluation. This is addressed through:

- Repair (removing assignments)
- Penalty (reducing fitness)
- Replacement (generating new feasible solutions)

2.1.3 Application Areas

- **Logistics:** Assigning shipments to transportation units while respecting capacity limits.
- **Resource Allocation:** Distributing limited computational or physical resources among competing tasks.
- **Scheduling:** Assigning jobs to machines with time/resource constraints.
- **Cloud Computing:** Task allocation to servers under capacity and performance constraints.
- **Manufacturing Systems:** Assigning operations to machines for optimal throughput.

2.2 Meta-heuristics Overview

2.2.1 Meta-heuristic Algorithms

Meta-heuristics are high-level optimization techniques designed to efficiently search large and complex solution spaces. They are particularly effective for NP-hard problems like GAP.

In this study, the following meta-heuristics are implemented:

- **BCGA (Binary-Coded Genetic Algorithm):** Uses binary encoding and genetic operators such as crossover and mutation.

- **RCGA (Real-Coded Genetic Algorithm):** Uses continuous representation with arithmetic crossover.
- **PSO (Particle Swarm Optimization):** Models collective behaviour using velocity and position updates.
- **TLBO (Teaching-Learning-Based Optimization):** Simulates teacher–student learning processes.
- **DE (Differential Evolution):** Uses vector-based mutation and crossover.
- **ABC (Artificial Bee Colony):** Mimics foraging behaviour of bees.

2.2.2 Motivation for Using Meta-heuristics

Exact optimization methods (e.g., branch-and-bound, dynamic programming) are computationally expensive for large GAP instances due to exponential growth in solution space.

Meta-heuristics provide:

- Near-optimal solutions in reasonable time
- Scalability to large datasets (e.g., gap12)
- Flexibility to handle complex constraints
- Ability to escape local optima through stochastic search

In this work, population-based meta-heuristics are preferred due to their parallel search capability and robustness.

One of the major challenges in applying meta-heuristics to GAP is maintaining feasibility under strict constraints. Since most algorithms operate in unconstrained search spaces, specialized techniques are required.

The following approaches are used in the implementations:

- **Repair Method:**
Modifies infeasible solutions by removing violating assignments until constraints are satisfied.
- **Penalty Method:**
Incorporates constraint violations into the objective function:
- **Replacement Method:**
Replaces infeasible solutions with newly generated feasible ones to maintain population quality.

3. Problem Definition

3.1 Problem Formulation

In this study, the GAP focuses on assigning a set of tasks to different agents in a way that maximizes total profit, while making sure that no agent exceeds its resource capacity. Each possible assignment of a task to an agent gives a certain profit but also uses up some of that agent's available resources.

The main goal is to find the best possible assignment of tasks to agents so that the overall profit is as high as possible, without violating any capacity constraints.

The problem is mathematically formulated as:

Objective Function:

$$\max \sum_{i=1}^m \sum_{j=1}^n p_{ij} x_{ij}$$

Subject to:

1.Task Assignment Constraint:

$$\sum_{j=1}^n x_{ij} \leq 1 \quad \forall i = 1, 2, \dots, m$$

2.Agent Capacity Constraint:

$$\sum_{i=1}^m r_{ij} x_{ij} \leq c_j \quad \forall j = 1, 2, \dots, n$$

3.Binary Decision Variables:

$$x_{ij} \in \{0, 1\}$$

3.2 Notations and Variable Definitions

The following notation is used throughout the study:

Symbol	Description
m	Number of tasks
n	Number of agents
p_{ij}	Profit obtained by assigning task i to agent j
r_{ij}	Resource consumption of task i on agent j
c_j	Capacity of agent j

Symbol	Description
x_{ij}	Capacity of agent j

4. Comparative Study of Meta-heuristic Techniques

4.1 Binary-Coded Genetic Algorithm (BCGA)

The Binary-Coded Genetic Algorithm (BCGA) works with a population of solutions represented in binary form, where each solution corresponds to a task - agent assignment. It improves solutions over generations using common genetic operations like tournament selection, crossover, and mutation.

In this implementation, constraint handling is done using repair and penalty methods. If a solution violates capacity constraints, it is either fixed (repair) or penalized in its fitness value. Since BCGA uses binary encoding, it fits naturally with discrete problems like GAP.

4.2 Real-Coded Genetic Algorithm (RCGA)

The Real-Coded Genetic Algorithm (RCGA) is an extension of the traditional GA where solutions are represented using continuous values between 0 and 1 instead of binary strings. These values are later converted into binary assignments using a threshold. RCGA uses arithmetic crossover and Gaussian mutation, which allow smoother and more flexible changes in solutions compared to BCGA.

After decoding, constraint handling is applied using either repair or penalty methods to ensure feasibility. Because of its real-valued encoding, RCGA explores the search space more effectively and can make finer adjustments to solutions. This makes it generally more suitable for GAP than BCGA, as it often leads to faster convergence and better solution quality, although the decoding step can sometimes introduce small inaccuracies.

4.3 Particle Swarm Optimization (PSO)

Particle Swarm Optimization (PSO) is a population-based method inspired by the way groups like birds or fish move together while searching for food. In this approach, each particle represents a possible solution and updates its position by learning from its own best solution (pbest) and the best solution found by the entire group (gbest).

In this implementation, particles are stored as real-valued vectors, which are later converted into binary solutions using a threshold. The velocity update uses inertia, cognitive, and social components, helping the algorithm balance exploration of new solutions and exploitation of good ones. Feasibility is maintained using repair or penalty methods.

PSO works well for GAP because it converges quickly and can efficiently search the solution space. However, one limitation is that it may converge too early and lose diversity, especially in later stages of the search.

4.4 Teaching-Learning-Based Optimization (TLBO)

Teaching-Learning-Based Optimization (TLBO) is inspired by how students learn in a classroom. It works in two main steps: in the teacher phase, all solutions move closer to the best one (like students learning from a teacher), and in the learner phase, solutions improve by interacting with each other.

In this implementation, solutions are stored as real-valued vectors and then converted into binary assignments for the GAP. Any constraint violations are handled using repair or penalty methods. Since TLBO uses real-valued encoding, it allows smoother updates and better exploration of the search space.

Overall, TLBO works well for GAP because it maintains a good balance between exploration and exploitation, requires very little parameter tuning, and shows stable convergence across different problem sizes.

4.5 Differential Evolution (DE)

Differential Evolution (DE) is a population-based optimization technique that creates new solutions by combining differences between randomly chosen individuals. It works by applying mutation and crossover to generate new candidate solutions (trial vectors), which are then compared with existing ones and selected based on their fitness.

In this implementation, solutions are represented as real-valued vectors, which are later converted into binary form for evaluation. Different constraint handling methods - repair, penalty, and replacement - are used, making the approach flexible for maintaining feasibility.

Because of its real-valued encoding, DE can explore the search space more effectively. It performs well for GAP as it is robust, maintains diversity in the population, and reduces the chances of getting stuck in poor solutions, making it one of the stronger methods in this study.

<<<

4.6 Artificial Bee Colony (ABC)

The Artificial Bee Colony (ABC) algorithm is inspired by how honey bees search for food. It works through three main phases - employed bees, onlooker bees, and scout bees - each contributing to exploring and improving solutions. In this approach, solutions are represented as real-valued vectors, which are later converted into binary form using a threshold.

To handle infeasible solutions, strategies like repair, penalty, or replacement are applied. One of the key strengths of ABC is its strong exploration ability, mainly due to the probabilistic selection of solutions and the role of scout bees, which help the algorithm escape local optima. However, compared to other algorithms, ABC tends to converge more slowly, which can affect its performance, especially on larger GAP instances.

5. Experimental Analysis

5.1 Parameter Settings

The performance of the implemented meta-heuristic algorithms depends significantly on the choice of control parameters. In this study, a uniform experimental setup is maintained across all GAP instances (*gap8*, *gap10*, *gap12*) to ensure fair comparison.

	BCGA	RCGA	PSO	TLBO	DE	ABC
Pop_Size	100	100	100	100	100	100
Generations	800	800	800	800	800	800
Crossover rate	0.8	0.8	--	--	0.8	--
Mutation Rate	0.15	0.15	--	0.15	0.15	--
Inertia weight	--	--	0.4	--	--	--
c1, c2	--	--	0.3, 0.3	--	--	--
Elite fraction	1 individual	1 individual	--	--	--	--
Scout Limit	--	--	--	--	--	100(=pop size)
No. of runs	20	20	20	20	20	20
Encoding Used	Integer assignment vector	Real-valued vector (float $\in [0,m)$), decoded via floor()	Integer assignment vector	Integer assignment vector	Integer assignment vector	Integer assignment vector

```

POP_SIZE      = 100      (>= 100)
ITERATIONS    = 800      (700-800)
CROSSOVER_RATE = 0.8      (0.7 or 0.8)
MUTATION_RATE = 0.15     (0.1-0.2)
SEED          = 42
NUM_RUNS      = 20

```

- For TLBO, a gene-wise greedy learning strategy is used where each gene of a learner is updated to match the teacher's gene only if the teacher's assignment gives higher profit for that job.

5.2 Constraint Handling and Variation Strategy

In GAP, handling constraints is extremely important because not every generated solution

adheres to the capacity limits. This research employs three primary methods across various algorithms to manage infeasible solutions:

- **Initial Population Feasibility:**

The basic design choice for this implementation is that the first population is set up to work perfectly from generation/iteration zero for all six algorithms. Instead of making random solutions and fixing them later with constraint handling, each initial individual is made using a greedy sequential construction heuristic:

- To make sure that the population is diverse, the order of jobs is changed at random.
- The agent with the most capacity and the most profit is given each job.
- If no agent has enough capacity for a job (which happens in tight cases like GAP-12), the agent with the least work is chosen as a backup.
- Then, the repair operator is used on any remaining capacity violations that were caused by the fallback step. This makes sure that every person in the initial population is fully feasible.

- **Penalty-Based Handling:**

Instead of throwing away infeasible solutions, they are punished by having their fitness value lowered by a violation cost that is based on how much capacity they violate. The penalty value is automatically set to be higher than the maximum possible total profit. This makes sure that no infeasible solution can ever get a higher score than a feasible one during selection. This lets the algorithm keep exploring different areas of the search space while still strongly favoring areas that are possible. After each update, repair is also done to make sure that there are always possible solutions available for tracking the best solution.

- **Repair-Based Handling:**

Infeasible solutions are changed right away to make them feasible again. If an agent's workload becomes too much, jobs are given to agents who still have enough capacity. The repair strategy is greedy; it moves jobs with the lowest profit contribution first and keeps high-value assignments wherever it can. In the population, the repaired solution takes the place of the original.

- **Feasibility Check:**

The `is_feasible()` function checks each solution against all agent capacity constraints after each step of handling constraints. It tells you if the solution is feasible, how much each agent is using resources, and which agents have been violated. When tracking and reporting the best solution found, only solutions that are possible (with no penalty) are taken into account.

In addition to repair, another strategy used instead of modification is replacement:

- **Replacement-Based Handling:**

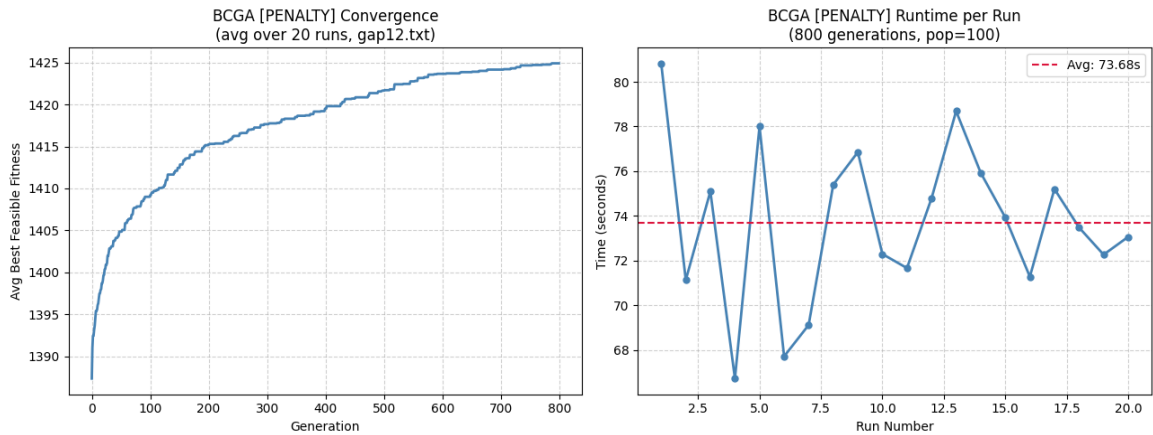
Instead of repairing an infeasible solution, a new one is made from scratch using a greedy sequential assignment. This means that each job is given to the agent with the most capacity and the most profit, in a random order to make sure there is a mix of people. If there isn't a suitable agent for a job during construction, the agent with

the least work is chosen and repairs are made later to fix any remaining problems.

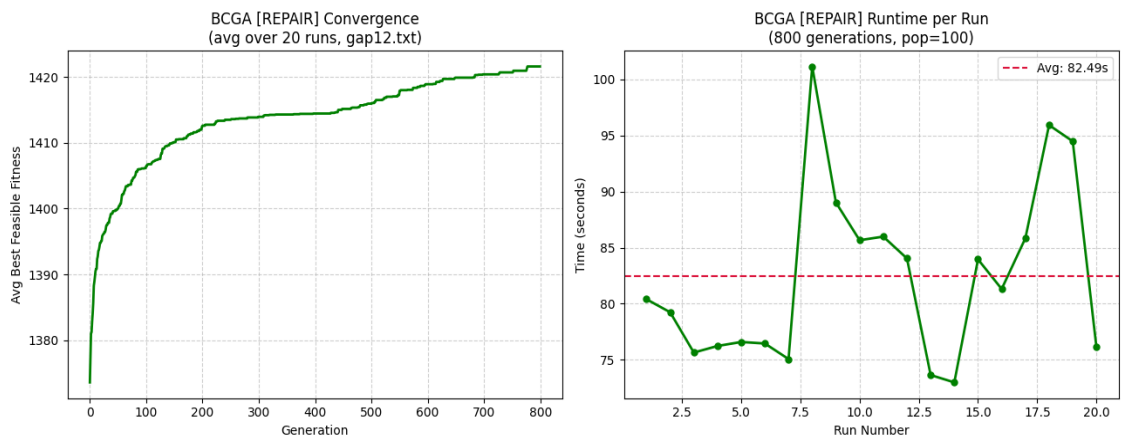
5.3 Results and Discussion

➤ Convergence curves (fitness vs. iterations)

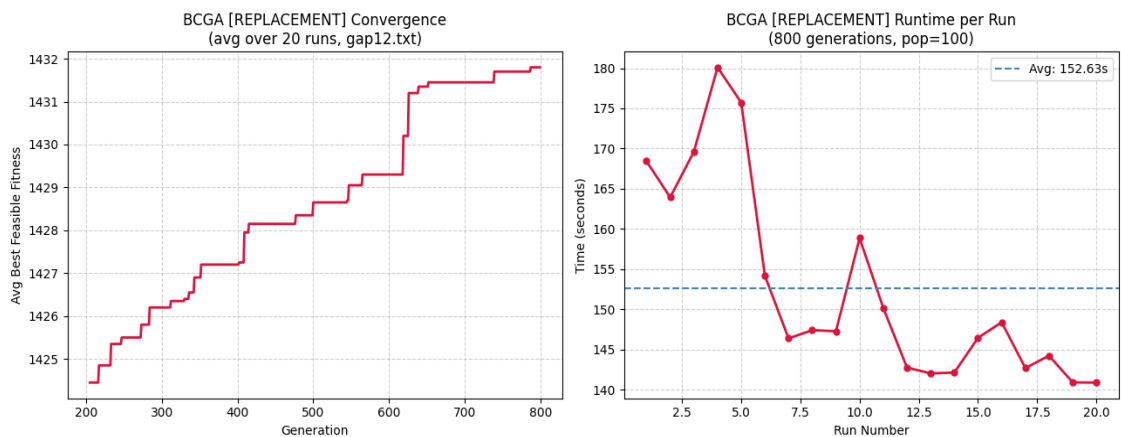
1. BCGA using Penalty Method



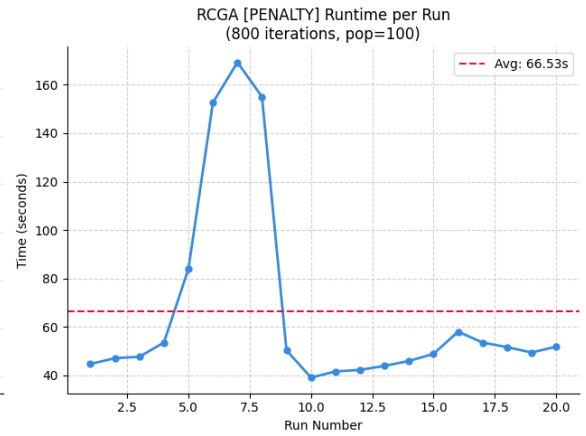
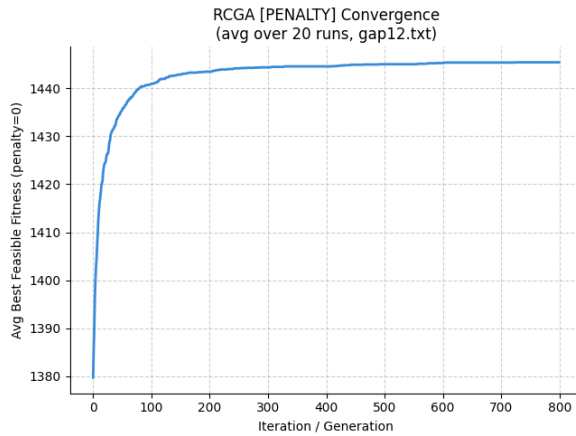
2. BCGA using Repair Method



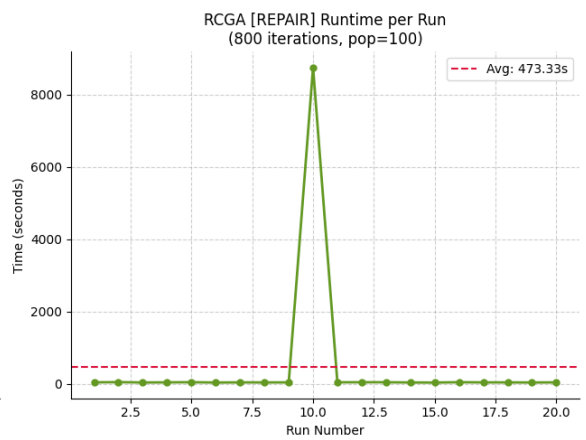
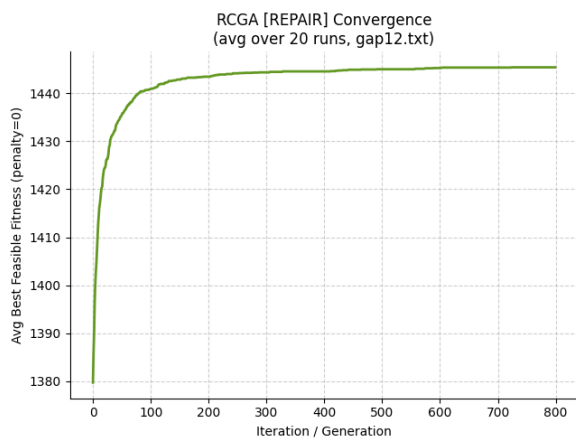
3. BCGA using Replacement Method



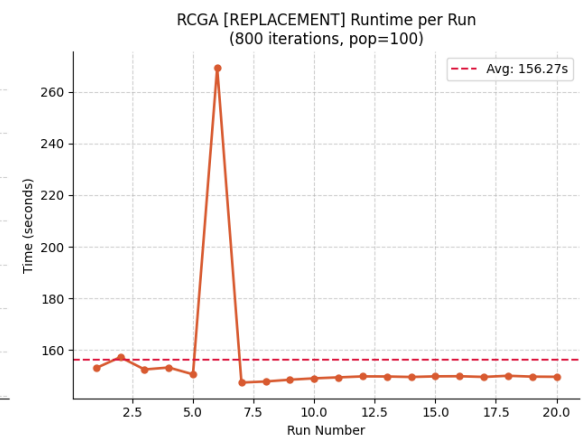
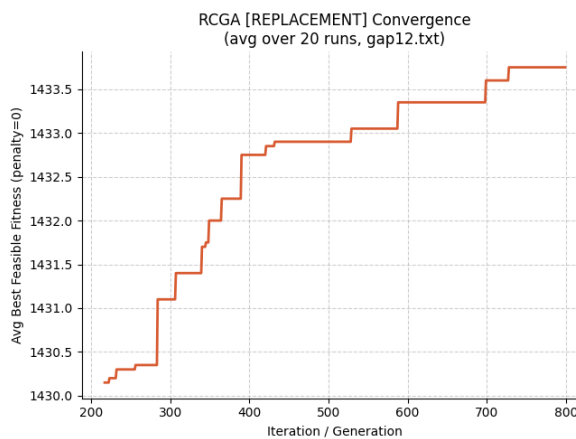
4. RCGA using Penalty Method



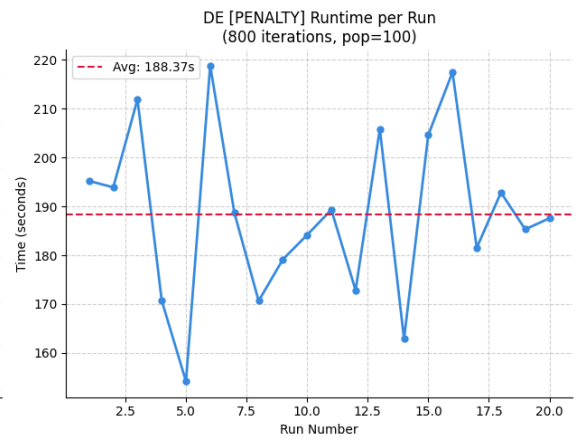
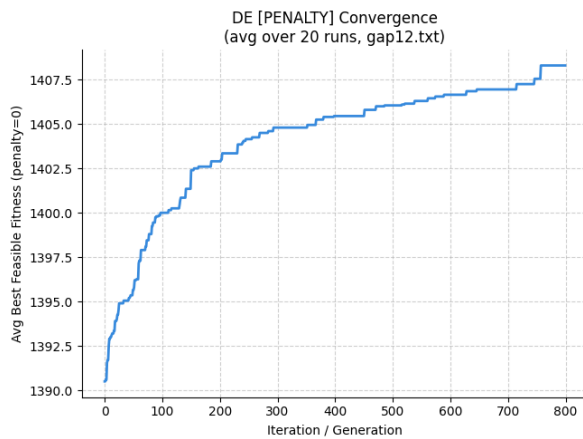
4. RCGA using Repair Method



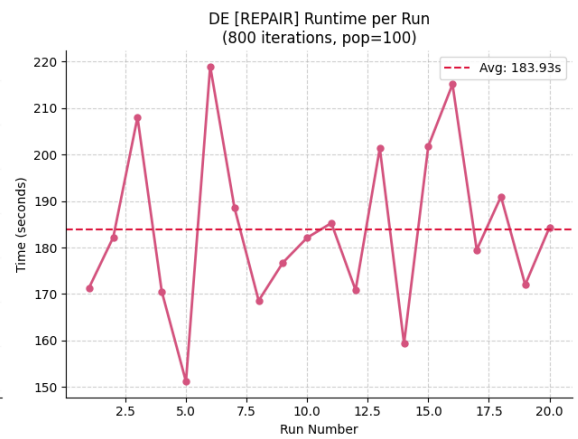
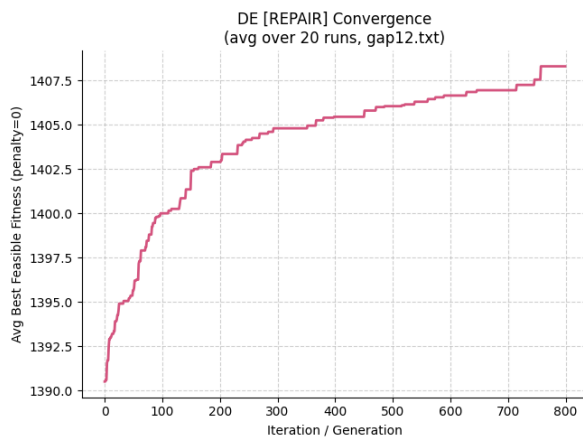
6. RCGA using Replacement Method



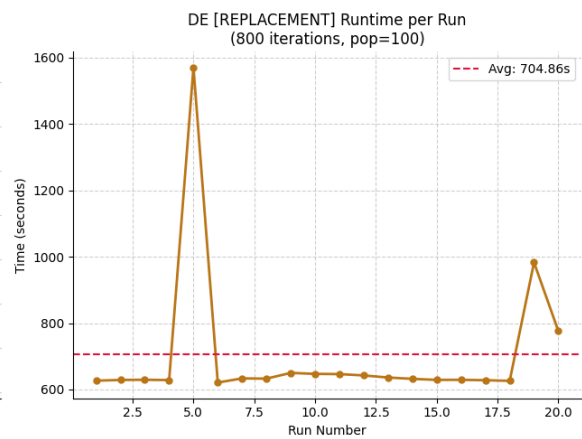
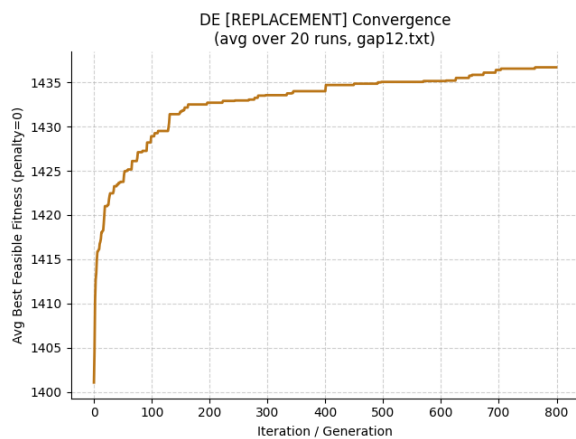
7. DE using Penalty Method



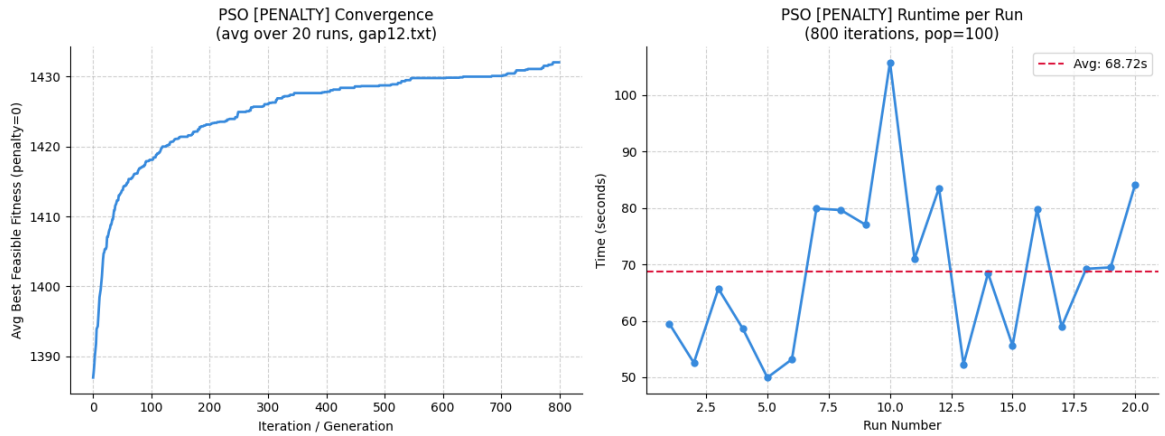
8. DE using Repair Method



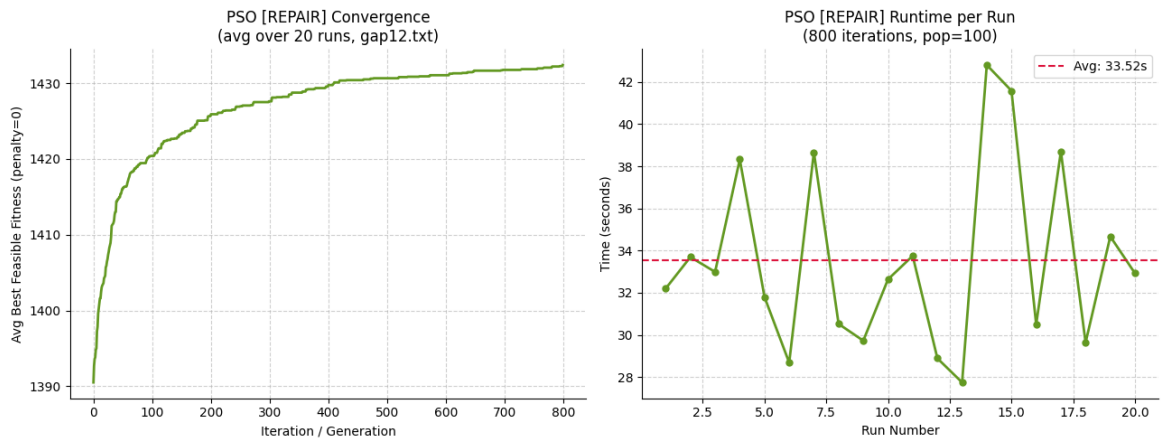
9. DE using Replacement Method



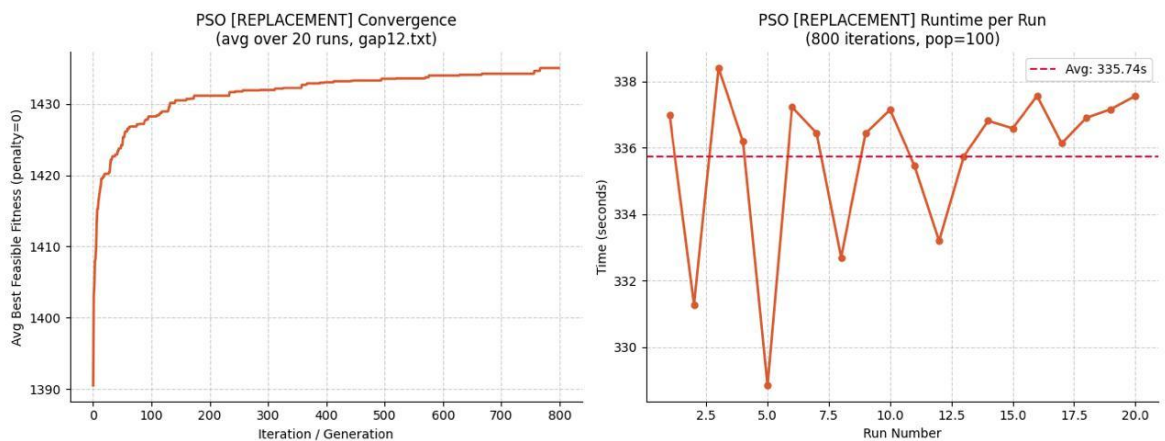
10. PSO using Penalty Method



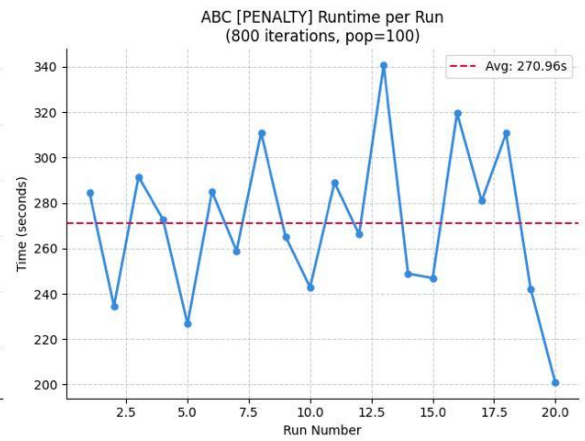
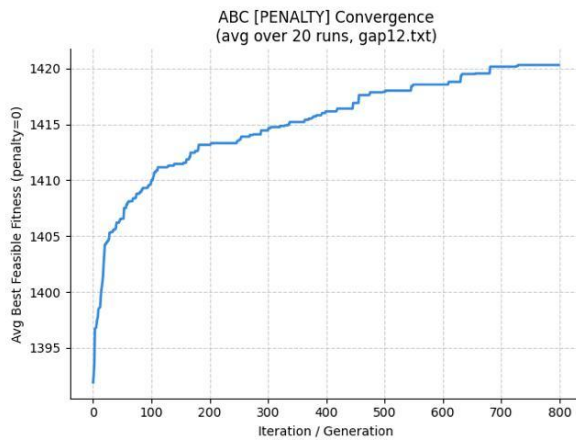
11. PSO using Repair Method



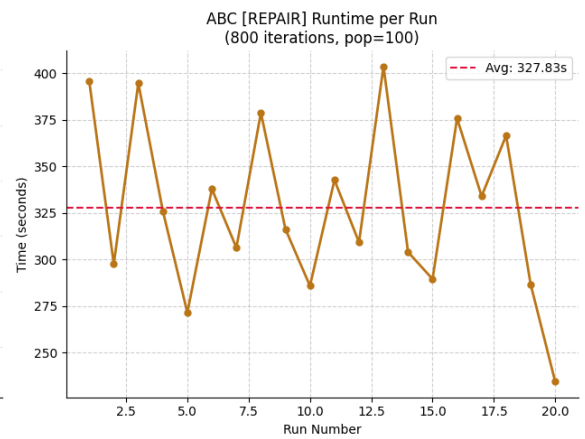
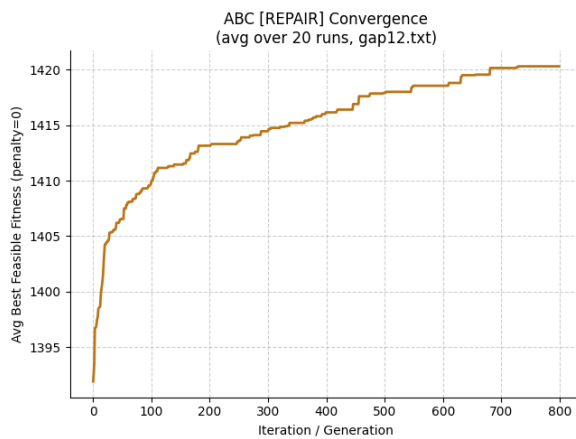
12. PSO using Replacement Method



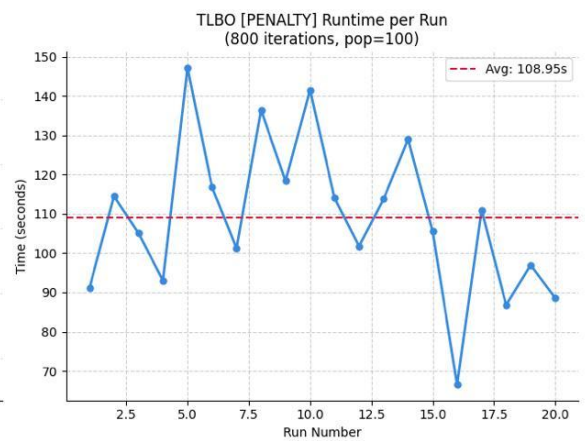
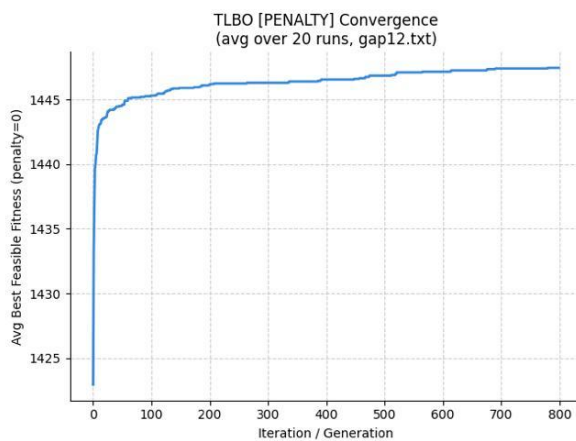
13. ABC using Penalty Method



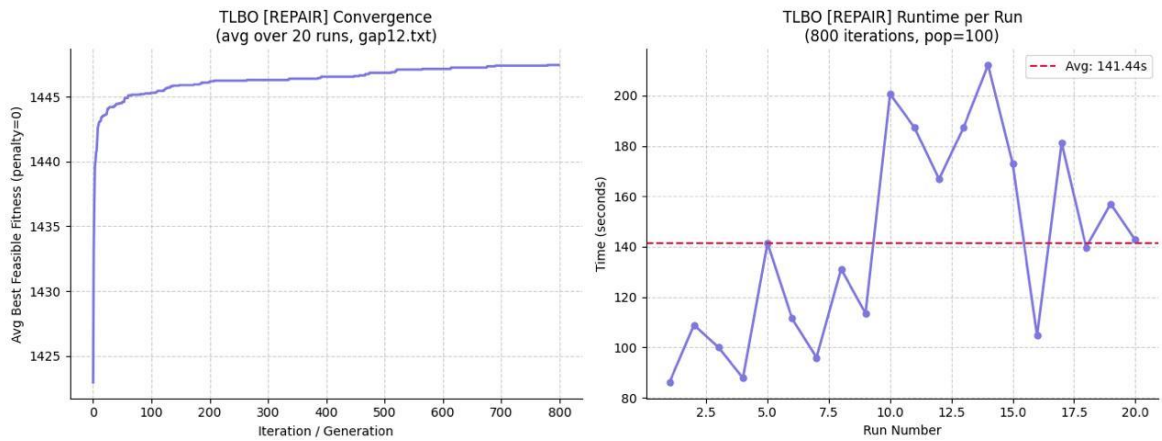
14. ABC using Repair Method



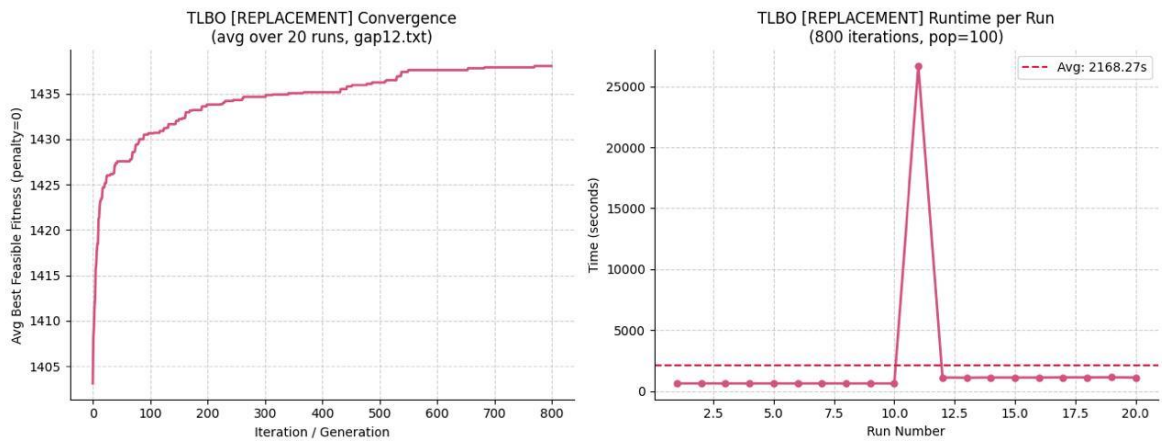
15. TLBO using Penalty Method



16. TLBO using Repair Method



17. TLBO using Replacement Method



➤ **Best Solution Found**

//Output of BCGA using Penalty

```

=====
Generalized Assignment Problem – BCGA [PENALTY]
=====
File           : gap12.txt
Problem sets   : 5
Agents         : 10   Jobs : 60
Runs           : 20
Generations     : 800
Population size : 100
Crossover rate  : 0.8
Mutation rate   : 0.15
Penalty per unit : 1460 (> max profit 1459, guarantees feasible best)
Constraint method : penalty
-----
Run 1 | Best Fitness : 1428 | Time : 80.83s
Run 2 | Best Fitness : 1417 | Time : 71.13s
Run 3 | Best Fitness : 1424 | Time : 75.10s
Run 4 | Best Fitness : 1427 | Time : 66.73s
Run 5 | Best Fitness : 1419 | Time : 78.00s
Run 6 | Best Fitness : 1425 | Time : 67.72s
Run 7 | Best Fitness : 1432 | Time : 69.13s
Run 8 | Best Fitness : 1426 | Time : 75.40s
Run 9 | Best Fitness : 1422 | Time : 76.85s
Run 10 | Best Fitness : 1423 | Time : 72.30s
Run 11 | Best Fitness : 1425 | Time : 71.67s
Run 12 | Best Fitness : 1431 | Time : 74.79s
Run 13 | Best Fitness : 1419 | Time : 78.69s
Run 14 | Best Fitness : 1431 | Time : 75.93s
Run 15 | Best Fitness : 1426 | Time : 73.93s
Run 16 | Best Fitness : 1422 | Time : 71.28s
Run 17 | Best Fitness : 1421 | Time : 75.21s
Run 18 | Best Fitness : 1428 | Time : 73.49s
Run 19 | Best Fitness : 1428 | Time : 72.27s
Run 20 | Best Fitness : 1424 | Time : 73.07s
-----
Summary
Best Fitness      : 1432
Average Fitness   : 1424.90

```

```

-----
Summary
Best Fitness      : 1432
Average Fitness   : 1424.90
Worst Fitness     : 1417
Std Deviation     : 4.06
-----
Runtime Summary
Total time (20 runs) : 1473.59s
Avg time per run     : 73.68s
Min run time         : 66.73s
Max run time         : 80.83s
-----
Feasibility Check on Best Solution
Feasible : YES - All constraints satisfied
All 10 agent capacity constraints satisfied.
Penalty in best solution : 0 (pure profit reported)
-----

```

//Output of PSO using Repair

```

=====
Generalized Assignment Problem – PSO [REPAIR]
=====
File           : gap12.txt
Problem sets   : 5
Agents         : 10   Jobs : 60
Runs           : 20
Iterations/Gens : 800
Population size : 100
Crossover rate  : 0.8
Mutation rate   : 0.15
Constraint method : repair
=====
Run  1 | Best Fitness :    1424 | Time : 32.20s
Run  2 | Best Fitness :    1436 | Time : 33.69s
Run  3 | Best Fitness :    1432 | Time : 32.99s
Run  4 | Best Fitness :    1427 | Time : 38.33s
Run  5 | Best Fitness :    1432 | Time : 31.80s
Run  6 | Best Fitness :    1431 | Time : 28.69s
Run  7 | Best Fitness :    1431 | Time : 38.66s
Run  8 | Best Fitness :    1433 | Time : 30.52s
Run  9 | Best Fitness :    1431 | Time : 29.73s
Run 10 | Best Fitness :    1430 | Time : 32.64s
Run 11 | Best Fitness :    1434 | Time : 33.75s
Run 12 | Best Fitness :    1429 | Time : 28.89s
Run 13 | Best Fitness :    1435 | Time : 27.75s
Run 14 | Best Fitness :    1430 | Time : 42.80s
Run 15 | Best Fitness :    1433 | Time : 41.58s
Run 16 | Best Fitness :    1439 | Time : 30.50s
Run 17 | Best Fitness :    1435 | Time : 38.70s
Run 18 | Best Fitness :    1435 | Time : 29.63s
Run 19 | Best Fitness :    1436 | Time : 34.67s
Run 20 | Best Fitness :    1435 | Time : 32.92s
=====
Summary
  Best Fitness      : 1439
  Average Fitness   : 1432.40

```

```

Run 19 | Best Fitness :    1436 | Time : 34.67s
Run 20 | Best Fitness :    1435 | Time : 32.92s
=====
Summary
  Best Fitness      : 1439
  Average Fitness   : 1432.40
  Worst Fitness     : 1424
  Std Deviation     : 3.38
=====
Runtime Summary
  Total time (20 runs) : 670.48s
  Avg time per run     : 33.52s
  Min run time        : 27.75s
  Max run time        : 42.80s
=====
Feasibility Check on Best Solution
  Feasible : YES – All constraints satisfied
  All 10 agent capacity constraints satisfied.
  Penalty in best solution : 0 (pure profit reported)
=====
Best Assignment (job => agent):

```

//Output of TLBO using Penalty

=====

Generalized Assignment Problem – TLBO [PENALTY]

=====

File : gap12.txt
 Problem sets : 5
 Agents : 10 Jobs : 60
 Runs : 20
 Iterations/Gens : 800
 Population size : 100
 Crossover rate : 0.8
 Mutation rate : 0.15
 Constraint method : penalty
 Penalty (auto) : 1460 (> max profit, guides teacher/learner selection)

Run 1	Best Fitness :	1445	Time :	91.14s
Run 2	Best Fitness :	1450	Time :	114.46s
Run 3	Best Fitness :	1449	Time :	105.08s
Run 4	Best Fitness :	1450	Time :	93.02s
Run 5	Best Fitness :	1445	Time :	147.15s
Run 6	Best Fitness :	1448	Time :	116.83s
Run 7	Best Fitness :	1450	Time :	101.26s
Run 8	Best Fitness :	1445	Time :	136.41s
Run 9	Best Fitness :	1445	Time :	118.35s
Run 10	Best Fitness :	1444	Time :	141.52s
Run 11	Best Fitness :	1449	Time :	113.99s
Run 12	Best Fitness :	1451	Time :	101.78s
Run 13	Best Fitness :	1448	Time :	113.76s
Run 14	Best Fitness :	1447	Time :	128.95s
Run 15	Best Fitness :	1448	Time :	105.52s
Run 16	Best Fitness :	1450	Time :	66.54s
Run 17	Best Fitness :	1445	Time :	110.99s
Run 18	Best Fitness :	1446	Time :	86.78s
Run 19	Best Fitness :	1447	Time :	96.97s
Run 20	Best Fitness :	1447	Time :	88.57s

Summary

Run 18	Best Fitness :	1446	Time :	86.78s
Run 19	Best Fitness :	1447	Time :	96.97s
Run 20	Best Fitness :	1447	Time :	88.57s

Summary

Best Fitness : 1451
 Average Fitness : 1447.45
 Worst Fitness : 1444
 Std Deviation : 2.11

Runtime Summary

Total time (20 runs) : 2180.88s
 Avg time per run : 108.95s
 Min run time : 66.54s
 Max run time : 147.15s

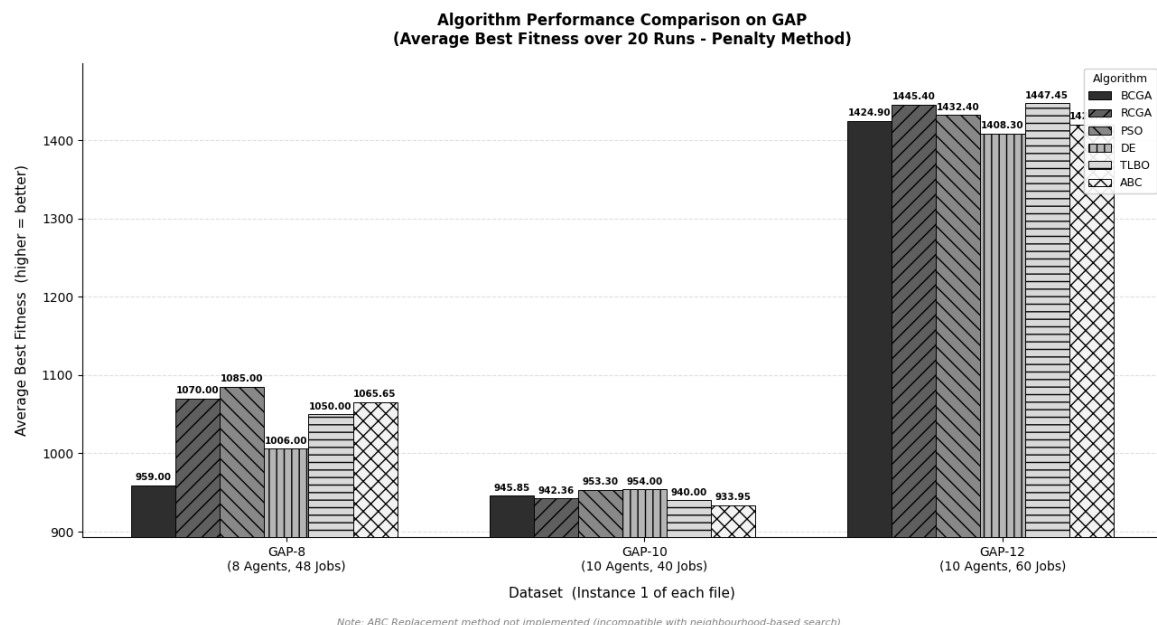
Feasibility Check on Best Solution

Feasible : YES – All constraints satisfied
 All 10 agent capacity constraints satisfied.
 Penalty in best solution : 0 (pure profit reported)

Algorithm	Repair	Penalty	Replacement
GA (Binary)	1431	1432	1440
GA (Real)	1449	1449	1443
PSO	1439	1439	1443
TLBO	1451	1451	1444
DE	1431	1431	1443
ABC	1431	1431	----

- TLBO had the best fitness of 1451 for both repair and penalty methods, making it the best overall algorithm. This shows that TLBO is good at systematic teacher-led exploitation.
- RCGA came in second with 1449, both for repair and penalty. This shows that using real-valued encoding with SBX crossover and polynomial mutation works very well for this kind of problem.
- PSO got 1443 points for replacement and 1439 points for repair and penalty. The swarm has better spatial coverage when it discards particles that don't work and starts over.
- BCGA showed the most improvement with replacement (1440) compared to repair (1431) and penalty (1432). This suggests that creating a brand new feasible individual when infeasibility happens adds more diversity to the population than fixing the existing one for binary-coded representations.
- DE got 1443 under replacement and 1431 under both repair and penalty, which is similar to PSO. Replacement helps DE by giving it new candidate solutions that the guided mutation strategy can then use effectively.
- Under both the repair and penalty methods, ABC got 1431. Replacement was not implemented for ABC because it did not work with neighborhood-based search.

➤ Average final fitness comparison



The histogram shows how well the algorithm worked on GAP-8, GAP-10, and GAP-12 by showing the average best fitness over 20 runs. PSO works best on GAP-8 (1085.00), but DE (954.00) and PSO (953.30) both do well on GAP-10, where the differences between algorithms are small. TLBO has the best fitness for the bigger

dataset (1447.45), followed closely by RCGA and PSO. This shows that it can handle larger datasets well.

In general, real-coded and swarm-based algorithms (RCGA, PSO, TLBO) do better than BCGA. The differences in performance become more clear as the size of the problem grows.

➤ Runtime Comparison

Algorithm	Repair (sec)	Penalty (sec)	Replacement (sec)
GA (Binary)	1649.84	1473.59	3052.56
GA (Real)	9466.87	1330.71	3125.69
PSO	670.48	1956.67	6714.77
TLBO	2828.87	2180.88	43365.49
DE	3678.71	3767.52	17544.60
ABC	6556.75	5419.15	---

5.4 Discussion

TLBO shows great robustness by getting the best score (1451) with both repair and penalty methods. RCGA also does consistently well (1449), which shows that real encoding is better than binary GA. DE and BCGA get better with replacement, but PSO stays the same across all methods. Repair and penalty yield nearly identical outcomes, suggesting comparable efficacy. In general, real-coded and swarm-based methods work better than BCGA because they are more consistent and can handle more data.

Table 3: Summary Comparison of Algorithms

Algorithm	Best Strategy	Convergence	Strength	Weaknesses
GA (Binary)	Replacement	Moderate	Simple, stable results	Lower fitness across all datasets
GA (Real)	Penalty/Repair	Fast	Better performance than BCGA; good search capability	Slightly lower than top methods; parameter sensitive

Algorithm	Best Strategy	Convergence	Strength	Weaknesses
PSO	Penalty/Repair	Moderate	Consistently high performance; best on GAP-8; strong stability	May depend on parameter tuning
TLBO	Penalty/Repair	Fast	Best performance on large dataset (GAP-12); parameter-free	Minor variability across runs
DE	Replacement	Moderate	Competitive performance (best on GAP-10); good global search	Slightly slower convergence on larger problems
ABC	Penalty/Repair	Slow–Moderate	Good exploration (diversity)	Slightly lower and less consistent performance, especially on GAP-10

6. Conclusion

The aim of this study was to evaluate different meta-heuristic algorithms for addressing the Generalized Assignment Problem through various constraint-handling techniques. The results show that good handling of feasibility, especially through repair and replacement methods, is necessary for getting high-quality solutions.

The experimental results for gap12 show that TLBO has the highest fitness (1451), which shows that it is very robust. RCGA also does very well (1449), which shows that real-coded encoding works well. PSO works well all the time, while DE and BCGA get better when they are replaced but are still not as good as PSO. ABC works moderately well but is only useful in certain situations. In general, TLBO and RCGA give the best quality solutions, while PSO strikes a good balance between performance and stability.

It can be concluded that RCGA and TLBO provide the best overall combination of solution quality and runtime, especially when applied to large-scale problems such as gap12.

Future research will expand upon the current findings through the use of hybrid techniques, optimal parameter tuning, and better initialization methods in order to expedite and further enhance convergence toward an optimal solution.

